

Predicting Enterprise Server Failure Across Global Data Centers

Problem Understanding

The firm maintains infrastructure for 10 enterprise customers globally, each having multiple data centers and over 5000 servers that constantly log monitoring data.

The telemetry data that can be gathered include:

- Memory utilization
- Disk I/O
- Latency
- Error messages
- Crash reports
- Past maintenance reports

The goal is to create an AI-based predictive monitoring system capable of:

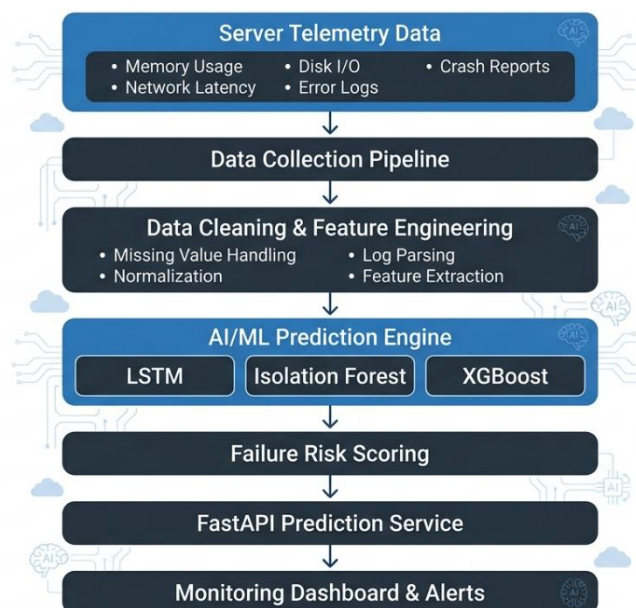
1. Downtime reduction by at least 40%
2. Applicability for different environments
3. Self-adjusting based on newly added infrastructure

Other constraints are as follows:

- Missing logs up to 25%
- Prediction time of less than 5 seconds per server

The suggested approach concentrates on predicting failures proactively.

Proposed Workflow



System Architecture

The following main building blocks make up the proposed system:

A. Data Collection Layer

Telemetry information is continuously collected from enterprise servers like the following:

- CPU usage
- Memory utilization
- Disk activity
- Network latency
- Error logs
- Crash reports

The information is aggregated into centralized monitoring storage.

B. Data Preprocessing Layer

Logs produced by infrastructure are often inconsistent and noisy. Hence, preprocessing is necessary before using data for machine learning.

Preprocessing Steps

- Dropping duplicates
- Missing value handling
- Normalization of telemetry metrics
- Parsing logs and feature extraction
- Creating time series windows

Handling Missing Logs

As many as 25% of the logs can be missing, and therefore the following techniques would be used:

- Interpolation
- Rolling average
- Statistical imputation

C. AI/ML Prediction Engine

The proposed AI engine uses different approaches to perform machine learning.

Model	Reason
LSTM	Handles sequential telemetry patterns
XGBoost	Fast and accurate structured-data prediction
Isolation Forest	Detects unknown anomalies

LSTM Network

LSTM networks are used for

- sequential telemetry processing,
- memory spike detection,
- trend spotting for crash reports,
- identification of unseen temporal patterns.

Isolation Forest

Isolation Forest detects abnormalities of a server, including the following:

- latency spike,
- crashes,
- disk activity, and
- unseen anomalies.

XGBoost

XGBoost performs

- prediction of telemetry structured data
- fast inference

Failure Prediction Formula

The system predicts server failure probability using telemetry-based feature analysis.

$$P(\text{Failure}) = f(\text{Memory}, \text{DiskIO}, \text{Latency}, \text{CrashRate})$$

Where:

- Memory = RAM utilization
- DiskIO = Disk activity
- Latency = Network delay
- CrashRate = Application crash frequency

Analysis of Requirement Fulfillment

Requirement 1: Decrease Downtime of at least 40%

The suggested AI solution helps detect problems before the failure of servers takes place.

The methods that allow decreasing the downtime include:

- early detection of anomalies
- predictive maintenance
- automatic alerts
- migration of workloads

- proactive remediation
- real-time monitoring

The identification of the patterns helps infrastructure specialists avoid any outages.

Requirement 2: Operates in Different Client Environments

Different clients have different infrastructures and workloads.

The system operates in various environments by means of

- having a shared general model
- fine-tuning the model according to the client
- retraining the model periodically

This way, it is possible to combine generalization capabilities and the prediction precision.

Requirement 3: Ability to Adapt Automatically to New Infrastructure Configurations

The features of the system include:

- continual learning
- periodic retraining
- adaptive analysis of telemetry data

As soon as new infrastructure patterns appear, the AI model can learn again on new telemetry data.

Prediction Service Pipeline in Real Time

Prediction service is hosted through:

- Python
- FastAPI
- asynchronous APIs

FastAPI assists in:

- fast inference
- scalability of predictions
- and response latency less than 5 seconds.

Server gets information about:

probability of failure,

risk score,

server health categorization.

Prototype Implementation

Technologies Used

- Python
- Pandas
- Scikit-learn
- Google Colab

Prototype Objective

The prototype demonstrates:

- telemetry preprocessing,
- ML model training,
- and server failure prediction.

Sample Dataset Example

Server	Memory %	Disk I/O	Latency	Crash Count	Failure
S101	92	88	170	5	Yes
S102	45	30	40	0	No
S103	85	79	140	4	Yes
S104	50	35	45	0	No

Conclusion

The suggested artificial intelligence-enabled predictive maintenance system will employ:

- Machine Learning algorithms
- Anomaly Detection algorithms
- Telemetry Analytics
- A scalable back-end infrastructure

for proactive prediction of enterprise server failures.

Through the use of real-time monitoring and advanced machine learning algorithms, this system can minimize downtime and ensure scalability of infrastructure management for enterprises.

Output

[48]



```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = {
    'memory_usage': [92, 45, 85, 50, 96, 72, 88, 40],
    'disk_io': [88, 30, 79, 35, 91, 60, 84, 25],
    'network_latency': [170, 40, 140, 45, 190, 80, 150, 30],
    'crash_count': [5, 0, 4, 0, 6, 1, 5, 0],
    'failure': [1, 0, 1, 0, 1, 0, 1, 0]
}

df = pd.DataFrame(data)

X = df.drop('failure', axis=1)
y = df['failure']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)

print("Prediction Output:", predictions)
print("Model Accuracy:", accuracy)
```



```
... Prediction Output: [0 1]
Model Accuracy: 0.5
```

Intelligent IT Ticket Auto-Resolution System

Problem Understanding

The organization runs IT support services for several enterprises, getting hundreds of thousands of IT support tickets each month. Support tickets can be:

- Text descriptions
- Screen shots
- system logs

The aim is to design an intelligent ticket management system that can:

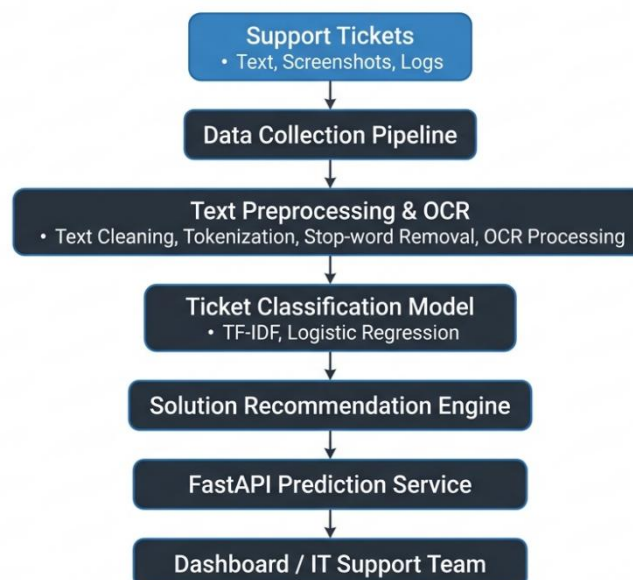
1. Automatically classifying support tickets
2. Suggesting solutions instantly

It must be able to deal with the following constraints:

- 30% noisy or poorly written tickets
- 10% tickets contain screenshots instead of text
- Response time < 2 seconds
- Classification accuracy $\geq 80\%$

This solution will leverage machine learning and NLP techniques in managing the process.

Proposed Workflow



System Architecture

The proposed system contains the following major components.

A. Ticket Collection Layer

Support tickets are collected from multiple enterprise clients in the system.

These tickets could include:

- issues with login
- crashes within software applications
- network related issues
- requests for access
- screenshots
- logs

Tickets from different enterprises are stored centrally for analysis.

B. Data Preprocessing Layer

IT support tickets are often messy and inconsistent.

Thus data preprocessing becomes essential prior to modeling.

Preprocessing steps include:

- removing noise
- converting to lowercase
- handling typos
- removal of stop words
- tokenizing
- extracting text using OCR for images

Handling Screenshot Based Tickets

As there are 10% tickets that could be in form of screenshots:

- OCR is utilized
- processed text is extracted
- keywords are identified

This helps in processing of tickets with screenshots.

C. Classification Engine for Tickets

A simple NLP and machine learning-based classification engine is suggested by us.

Model	Reason
TF-IDF	Converts ticket text into numerical vectors
Logistic Regression	Fast and effective text classification
OCR	Extracts text from screenshots

Use of TF-IDF:

- Converts textual data to numerical vectors
- Identifies important keywords in support tickets

Logistic Regression

Logistic Regression is applied to:

- Ticket classification
- Quick prediction
- Lightweight implementation
- Lower latency

Ticket Classification Formula

The ticket category prediction depends on extracted textual features.

$$P(\text{Category}) = f(\text{KeywordFeatures}, \text{ErrorLogs}, \text{OCRText})$$

Where:

- Keyword Features = Important ticket keywords
- ErrorLogs = System-generated logs
- OCRText = Extracted screenshot text

Requirements Fulfillment Analysis

Requirement 1: Automated Classification of Support Tickets

Support tickets are classified into:

- Login Error
- Networking Problems
- Application Crash
- User Access Requests

Using machine learning algorithms helps save efforts needed to manually sort tickets.

Requirement 2: Recommendation of Solutions

Upon classification of the tickets, predefined solutions are suggested by the system.

This helps minimize the response time and optimize IT support work.

Constraints Fulfillment Strategy

Dealing with Noisy Support Tickets

Due to the fact that some of the support tickets might have spelling errors or vague descriptions:

- text preprocessing,
- handling typos,
- keyword identification

Dealing with Support Tickets That Contain Screenshots

OCR algorithms help process screenshot-based tickets.

Keeping the Response Time Under 2 Seconds

ML models chosen for the task are lightweight, and FastAPI framework is deployed to achieve low latency.

Ensuring a Predicted Classifying Accuracy Over 80%

Labelled support tickets are fed to the training process to improve the prediction quality.

Prediction Service Pipeline

Ticket Prediction Service is built with:

- Python
- Scikit-learn
- FastAPI

The service does:

- ticket classification
- prediction of categories
- recommendation of solutions

in real-time.

Prototype Implementation

Used Technologies

- Python
- Pandas
- Scikit-learn
- Google Colab

Goal of the Prototype

The prototype illustrates:

- ticket processing
- vectorizing tickets
- ticket classification
- output prediction

Sample Ticket Dataset

Ticket	Category
Unable to login to system	Login Issue
Internet connection is slow	Network Issue
Application crashes frequently	Software Crash
Password reset not working	Login Issue

Conclusion

The suggested intelligent ticket resolution system incorporates:

- NLP preprocessing
- text extraction using OCR
- classification using machine learning algorithms
- and a lightweight back-end architecture

for enabling automated ticket processing for enterprises' IT support.

This could lead to increased operational efficiency due to automated classification and solution suggestions.

Output

[55]

✓ Os



```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
```

```
tickets = [
    "Unable to login to system",
    "Password reset not working",
    "Cannot access my account",

    "Internet connection is slow",
    "Network keeps disconnecting",
    "WiFi not working properly",

    "Application crashes frequently",
    "Software closes automatically",
    "System error while opening application"
]
```

```
categories = [
    "Login Issue",
    "Login Issue",
    "Login Issue",

    "Network Issue",
    "Network Issue",
    "Network Issue",

    "Software Crash",
    "Software Crash",
    "Software Crash"
]
```

```
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(tickets)
```

```
model = LogisticRegression()
model.fit(X, categories)
```

```
new_ticket = ["WiFi connection is unstable"]
new_ticket_vector = vectorizer.transform(new_ticket)
prediction = model.predict(new_ticket_vector)
```

```
print("Predicted Category:", prediction[0])
```



... Predicted Category: Network Issue